

Deskripsi Project

Nusantara Mon adalah game RPG 2D berbasis Python & Pygame yang menggabungkan nuansa klasik monster battle dengan kekayaan budaya Nusantara. Pemain berperan sebagai Pendekar Tamer yang menjelajahi dunia penuh legenda, bertemu tokoh sejarah, dan melawan monster mitologi khas Indonesia.

Anggota Kelompok

1. Viandika Rizky Ismono (25051204084)
2. Naufal Zidan Nur Zaki Ramadhani (25051204104)
3. Gabriel Renhard Yanengga (25051204249)
4. Andrean Nur Wahid (25051204250)

Fitur Utama

- Eksplorasi Peta Nusantara : Jelajahi area dengan NPC legendaris seperti Prabu Siliwangi, Nyai, Gajah Mada, hingga Kian Santang.
- Pertarungan Monster : Hadapi Buto Ijo, Leak-Mon, Cendrawasih, hingga Raja Jin dalam sistem battle berbasis giliran.
- Partner Monster : Pilih partner awal (Garuda-Mon atau Hanuman-Mon) dengan skill unik, lalu kembangkan level mereka.
- Sistem Quest & Item : Kumpulkan jamu tradisional untuk membantu perjalananmu.
- Efek Visual & Audio : Animasi partikel, minimap radar, transisi wipe, serta musik latar yang mendukung suasana Nusantara.
- NPC Interaktif : Dialog dengan tokoh Nusantara yang memberi misi, penyembuhan, atau hadiah item.

Cara Menjalankan Project

- A. Pemain menjalankan game dan masuk ke Menu Utama
- B. Saat di halaman utama, pemain akan melihat 5 opsi menu yaitu :
 1. Mulai Pertarungan : Memulai permainan baru
 2. Lanjutkan : Melanjutkan permainan yang telah disimpan
 3. Pengaturan : Mengatur konfigurasi game seperti Mode Game, Suara dan Ukuran layar

4. Keluar Game : Menutup permainan
5. Audio : Menghidupkan dan mematikan suara
- C. Pilih Mulai Pertarungan untuk masuk ke halaman Pemilihan Monster, kemudian pilih salah satu monster partner awal, yaitu Garuda atau Hanoman
- D. Setelah memilih monster, pemain akan masuk ke Map Permainan dan di dalam map terdapat :
 1. NPC yang dapat diajak berinteraksi
 2. Monster liar yang dapat dilawan
 3. Boss Raja Jin sebagai musuh utama
- E. Ketika pemain mendekati NPC, akan muncul dialog yang berisi informasi, bantuan, atau misi yang harus diselesaikan
- F. Ketika pemain mendekati monster liar, akan berpindah ke Mode Pertarungan dan player dapat menggunakan skill dan item heal untuk mengalahkan musuh. Jika pertarungan dimenangkan maka akan mendapatkan exp dan jika kalah maka player akan respawn ke tempat awal dan hp akan dipulihkan
- G. Ketika mendekati Boss Raja Jin, akan berpindah ke mode pertarungan. Disarankan monster berlevel ≥ 3 agar membuka ultimate agar kuat dalam melawan Boss Raja Jin
- H. Setelah berhasil mengalahkan boss Raja Jin, pemain dinyatakan telah menyelesaikan permainan

Penjelasan OOP

1. Inheritance (Pewarisan)

Inheritance merupakan konsep OOP yang memungkinkan suatu class mewarisi atribut dan method dari class lain. Pada project ini, class Entity digunakan sebagai parent class yang menyimpan atribut dan fungsi umum seperti posisi, animasi, dan gambar karakter.

Class Utama

class Entity:

```
def __init__(self, x, y, width, height, color, image_path=None):
    super().__init__()
    self.rect = pygame.Rect(x, y, width, height)
    self.color = color; self.image = None; self.image_path = image_path

    self.frames = None
```

```
self.direction = 0

self.anim_frame = 0

self.anim_speed = 80

self.last_update = pygame.time.get_ticks()

self.is_moving = False
```

Class Turunan

```
class OverworldMonster(Entity):
```

```
    def __init__(self, x, y, m_type, image_path=None):
        super().__init__(x, y, MONSTER_SIZE, MONSTER_SIZE, RED, image_path)
        self.m_type = m_type; self.base_y = y; self.anim_offset = random.randint(0, 100)
        ...
```

```
class NPC(Entity):
```

```
    def __init__(self, x, y, name, dialog_message, image_path="assets/npc.png"):
        super().__init__(x, y, NPC_SIZE, NPC_SIZE, YELLOW, image_path)
        self.name = name; self.message = dialog_message
        ...
```

```
class Monster(Entity):
```

```
    def __init__(self, name, x, y, color, element, hp, attack, image_path=None):
        super().__init__(x, y, 100, 100, color, image_path)
        self.name = name; self.element = element; self.level = 1; self.exp = 0
        ...
```

2. Encapsulation (Enkapsulasi)

Encapsulation merupakan proses menyembunyikan data dan mengontrol akses terhadap data tersebut. Pada project ini, atribut `_hp` tidak diakses secara langsung, melainkan melalui property dan setter.

```
@property
```

```
def hp(self):
```

```
    return self._hp
```

```
@hp.setter
```

```
def hp(self, value):
```

```
    self._hp = max(0, min(value, self.max_hp))
```

Penjelasan:

Nilai HP hanya dapat diubah melalui setter hp, sehingga nilainya selalu berada dalam rentang 0 hingga `max_hp`. Hal ini menjaga konsistensi data dan mencegah perubahan yang tidak valid.

3. Abstraction (Abstraksi)

Abstraction merupakan konsep menyembunyikan detail implementasi dan hanya menampilkan fungsi yang diperlukan kepada pengguna.

```
def take_damage(self, skill, attacker_attack, diff_multiplier=1.0):
```

```
    pemanggilan
```

```
    monster.take_damage(skill, attack)
```

Penjelasan:

Pengguna cukup memanggil method `take_damage()` tanpa perlu mengetahui proses di dalamnya, seperti perhitungan damage, critical hit, elemental multiplier, status effect, dan mekanisme lainnya. Seluruh proses tersebut disembunyikan dalam method tersebut.

4. Polymorphism (Polimorfisme)

Polymorphism merupakan kemampuan method yang sama untuk menghasilkan perilaku yang berbeda tergantung pada objek yang menggunakannya.

```
class Item:
```

```
    def use(self, target):
```

```
        pass
```

```
class HealthPotion(Item):
```

```
    def use(self, target):
```

```
        target.hp += self.heal_amount
```

```
class AttackPotion(Item):
```

```
    def use(self, target):
```

```
        target.base_attack += self.buff_amount
```

```
pemanggilan
```

```
item.use(player)
```

Penjelasan:

Method use() dipanggil dengan cara yang sama, tetapi menghasilkan perilaku yang berbeda tergantung jenis objeknya. Jika objek berupa HealthPotion, maka HP karakter akan bertambah. Jika objek berupa AttackPotion, maka nilai serangan karakter akan meningkat. Hal ini menunjukkan penerapan Polymorphism melalui Method Overriding.